Medical Computer Vision Lab

Designing and Scaling APIs/Endpoints (Flask, Django, FastAPI): Lab Standard

Maintainer: Ashery Mbilinyi

Version 1.0 : October 16, 2026

Contents

1 Purpose **3**

2 Core Concepts **3**
2.1 API and Endpoint . . . 3
2.2 Common ML Service Endpoints (Recommended) . . . 3
2.3 Contract First: Request/Response Schema . . . 3

3 Framework Selection Guide **3**
3.1 Flask . . . 3
3.2 Django (with Django REST Framework) . . . 3
3.3 FastAPI (Recommended Default) . . . 3

4 Scaling Concepts (What "Scaling an API" Means) **4**
4.1 Concurrency . . . 4
4.2 Horizontal Scaling . . . 4
4.3 Batching and Throughput (Critical for GPUs) . . . 4
4.4 Load Balancing . . . 4

5 Lab Policy **4**
5.1 Minimum Requirements for Any API Service . . . 4
5.2 Recommended Requirements for Production-Style Services . . . 4

6 Coding Examples **4**
6.1 FastAPI Example (Recommended Inference Service) . . . 4
6.1.1 app.py . . . 4
6.1.2 Run locally . . . 5
6.1.3 Test . . . 6
6.2 Flask Example (Minimal Prototype) . . . 6
6.3 Django REST Framework (DRF) Sketch . . . 6

7 Scaling Recipes **7**
7.1 Production Server (FastAPI) . . . 7
7.2 Reverse Proxy (Nginx) — Minimal Concept . . . 7
7.3 Kubernetes (High Level) . . . 7

8 Common Pitfalls and Guidance **7**
8.1 Async Misconceptions 7
8.2 Model Loading 7
8.3 Versioning 7

9 Checklist (Code Review) **8**

1 Purpose

This document defines MCV Lab standards for **designing** and **scaling** APIs/endpoints for ML systems. The goal is to ensure our services are:

- **Correct**: consistent request/response contracts and validation
- **Reliable**: robust error handling, health checks, and observability
- **Maintainable**: clear project structure and versioning
- **Scalable**: supports increased traffic and safe deployment patterns

2 Core Concepts

2.1 API and Endpoint

- **API**: the set of routes exposed by a service.
- **Endpoint**: a specific route (e.g., `POST /predict`, `GET /health`).

2.2 Common ML Service Endpoints (Recommended)

All ML inference services should implement:

- `GET /health`: process is alive (liveness)
- `GET /ready`: model is loaded and ready (readiness)
- `GET /version`: service version and model version
- `POST /predict`: inference on a single request
- `POST /batch_predict` (optional): batch inference for throughput

2.3 Contract First: Request/Response Schema

Each endpoint must have:

- a well-defined request schema (types, required fields, ranges)
- a well-defined response schema
- clear error responses with HTTP status codes

3 Framework Selection Guide

3.1 Flask

Best for: simple prototypes and lightweight services.
Tradeoff: minimal structure; you must add validation and documentation manually.

3.2 Django (with Django REST Framework)

Best for: full platforms with authentication, database models, admin UI, and complex workflows.
Tradeoff: heavier than needed for pure inference microservices.

3.3 FastAPI (Recommended Default)

Best for: modern inference microservices with typed schemas and automatic docs.
Why we like it: Pydantic validation + OpenAPI docs + clean contracts.

4 Scaling Concepts (What "Scaling an API" Means)

4.1 Concurrency

Ability to serve multiple requests at once. Common approaches:

- multiple worker processes (e.g., Gunicorn)
- async I/O (FastAPI) for network/disk-bound operations

Important: ML inference is typically CPU/GPU-bound; async does not magically speed up inference.

4.2 Horizontal Scaling

Run multiple replicas of the API service and load-balance across them.

4.3 Batching and Throughput (Critical for GPUs)

Batch inference improves throughput by using GPU efficiently. If latency constraints allow, prefer batching.

4.4 Load Balancing

Use a reverse proxy (Nginx) or platform load balancer (Kubernetes Service/Ingress).

5 Lab Policy

5.1 Minimum Requirements for Any API Service

- `/health`, `/ready`, `/version`, `/predict` implemented
- request validation present (FastAPI/Pydantic strongly preferred)
- structured logging (JSON or consistent text format)
- deterministic model versioning (model id/hash + git commit in `/version`)
- containerized deployment recipe (Dockerfile)

5.2 Recommended Requirements for Production-Style Services

- rate limiting (basic protection)
- authentication/authorization if exposing beyond local network
- metrics endpoint (Prometheus) or equivalent
- safe rollout and rollback strategy

6 Coding Examples

6.1 FastAPI Example (Recommended Inference Service)

6.1.1 app.py

```
from __future__ import annotations

import os
from typing import List, Optional

from fastapi import FastAPI, HTTPException
```

```python
from pydantic import BaseModel, Field

APP_VERSION = os.getenv("APP_VERSION", "dev")
MODEL_VERSION = os.getenv("MODEL_VERSION", "unknown")

app = FastAPI(title="MCV Inference API", version=APP_VERSION)

# ---- Schema (Contract) ----
class PredictRequest(BaseModel):
    # Example: a vector input. Replace with your real schema.
    x: List[float] = Field(..., min_length=1, description="Input feature vector.
    ")
    return_probs: bool = False

class PredictResponse(BaseModel):
    y: int
    probs: Optional[List[float]] = None
    model_version: str

# ---- Model loading (placeholder) ----
MODEL_LOADED = True

def dummy_model_predict(x: List[float]) -> tuple[int, List[float]]:
    # Replace with real model inference
    # Example rule:
    score = sum(x)
    y = 1 if score > 0 else 0
    probs = [1.0 - 0.2, 0.2] if y == 0 else [0.2, 0.8]
    return y, probs

# ---- Health endpoints ----
@app.get("/health")
def health():
    return {"status": "ok"}

@app.get("/ready")
def ready():
    if not MODEL_LOADED:
        raise HTTPException(status_code=503, detail="Model not loaded")
    return {"status": "ready"}

@app.get("/version")
def version():
    return {"app_version": APP_VERSION, "model_version": MODEL_VERSION}

# ---- Inference ----
@app.post("/predict", response_model=PredictResponse)
def predict(req: PredictRequest):
    if not MODEL_LOADED:
        raise HTTPException(status_code=503, detail="Model not loaded")

    y, probs = dummy_model_predict(req.x)

    return PredictResponse(
        y=y,
        probs=probs if req.return_probs else None,
        model_version=MODEL_VERSION,
    )
```

6.1.2 Run locally

```
pip install fastapi uvicorn pydantic
uvicorn app:app --host 0.0.0.0 --port 8000
```

6.1.3 Test

```
curl http://localhost:8000/health
curl http://localhost:8000/version

curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"x":[1.0,-0.5,0.2], "return_probs": true}'
```

6.2 Flask Example (Minimal Prototype)

```
from flask import Flask, request, jsonify

app = Flask(__name__)
MODEL_LOADED = True

@app.get("/health")
def health():
    return jsonify({"status": "ok"})

@app.get("/ready")
def ready():
    if not MODEL_LOADED:
        return jsonify({"status": "not_ready"}), 503
    return jsonify({"status": "ready"})

@app.post("/predict")
def predict():
    payload = request.get_json(force=True)
    x = payload.get("x")
    if x is None:
        return jsonify({"error": "Missing field: x"}), 400
    y = 1 if sum(x) > 0 else 0
    return jsonify({"y": y})
```

6.3 Django REST Framework (DRF) Sketch

Use DRF if you need user accounts, permissions, and database-backed workflows.

```
# serializers.py
from rest_framework import serializers

class PredictSerializer(serializers.Serializer):
    x = serializers.ListField(
        child=serializers.FloatField(), allow_empty=False
    )

# views.py
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
from .serializers import PredictSerializer

class PredictView(APIView):
    def post(self, request):
        s = PredictSerializer(data=request.data)
        s.is_valid(raise_exception=True)
        x = s.validated_data["x"]
        y = 1 if sum(x) > 0 else 0
```

```
        return Response({"y": y}, status=status.HTTP_200_OK)
```

7 Scaling Recipes

7.1 Production Server (FastAPI)

For production, do not rely on a single dev server. Use multiple workers.

```
pip install "uvicorn[standard]" gunicorn

gunicorn -k uvicorn.workers.UvicornWorker \
  -w 2 \
  -b 0.0.0.0:8000 \
  app:app
```

7.2 Reverse Proxy (Nginx) — Minimal Concept

Nginx can terminate TLS and load balance across multiple app workers/replicas.

7.3 Kubernetes (High Level)

Kubernetes scales by running multiple replicas:

- **Deployment**: defines desired replicas
- **Service**: load-balances traffic across replicas
- **HPA**: autoscaling based on CPU/custom metrics

8 Common Pitfalls and Guidance

8.1 Async Misconceptions

Async helps with I/O, not with GPU-bound inference. For GPU inference:

- use batching where possible
- consider a queue-based design for heavy requests
- avoid too many workers that compete for the same GPU memory

8.2 Model Loading

- load model once at startup (not per request)
- warm up the model (optional) to stabilize latency
- expose readiness only when model is loaded

8.3 Versioning

Always expose:

- git commit hash (app version)
- model artifact id/hash (model version)

9 Checklist (Code Review)

Before merging an API service:

- endpoints: `/health`, `/ready`, `/version`, `/predict`
- request/response schema is documented and validated
- errors return correct HTTP codes (400 vs 503 vs 500)
- model loads once and readiness reflects true state
- production run command documented (Gunicorn/Uvicorn)
- Dockerfile present